Async Swap: Coincidence-of-Wants Matching on Uniswap v4 (WIP)

Msaki, Meek Li, Jiasun

October 18, 2026

Abstract

Large swaps have no good home on-chain. A constant-product pool quotes a price that worsens with size, and the liquidity needed to absorb an institutional-size trade is rarely resident in one pool. We present Async Swap, a Uniswap v4 hook that turns an exact-input swap into a resting limit order with an on-chain order book and matching engine. A maker names an exact input and the minimum output it will accept; the input is escrowed in the PoolManager as ERC-6909 claims. The order then finds its counterparty asynchronously: it is matched against resting orders of the opposite side by coincidence of wants, tried against the pool's own liquidity when that satisfies the limit, and otherwise rests on the book until a later order crosses it, an independent filler settles it at the maker's price, or the maker cancels. The book is a Patricia radix tree of price levels in the hook's storage, so the best opposing price is found first; each pool chooses how same-price orders are sequenced, by arrival or by escrowed USDC priority bids that are donated to active liquidity providers as the order fills. Order-to-order matches carry no protocol fee. The escrow, conservation and priority properties of the design are machine-checked in Lean 4 and re-proved on every commit. The contracts are deployed as unaudited test deployments on Arc and Unichain ahead of an official launch.

1 Introduction

Automated market makers price a trade by moving along a curve. For Uniswap v3 [1] and v4 [2] that curve is a constant product between concentrated liquidity positions, and the price a trader receives degrades continuously with the size of the trade. For a trade that is small relative to the liquidity in range this is a feature: the pool always quotes, and the cost of immediacy is a few basis points. For a trade that is large relative to that liquidity it is the whole problem. The quote moves against the trader, the depth is not there, and whatever is there is visible to searchers who can position around the trade before it lands.

Off-chain venues solve this with time. A limit order rests at the price its owner chose until a counterparty arrives, and a block trade is negotiated rather than swept through a curve. On-chain, that pattern has mostly been imported as off-chain order flow with on-chain settlement: a signed intent is broadcast to solvers or fillers who compete to execute it, and the chain sees only the final trade. This works, but the order itself lives off-chain, in a relayer or a solver's inbox, and the trader depends on that infrastructure to be honest and available.

Async Swap keeps the order on-chain. The core observation is that a Uniswap v4 hook can take an exact-input swap away from the pool: the `beforeSwap` callback can consume the whole input as a custom accounting delta, escrow it as claims held by the hook, and record an order instead of executing a swap. From that point the order is a first-class object in contract storage, with a price, a size, an owner, and an event history that anyone can index. What the hook does with that order is the subject of this paper.

Our contributions are as follows.

- An *asynchronous limit order* on Uniswap v4: an exact-input swap whose input is escrowed and whose output is settled later, at or above a maker-chosen minimum, by whoever can meet

it (Section 3).

- An *on-chain order book*: a Patricia radix tree of price levels per pool that locates the best opposing price in storage, with a crossing rule that is exact at equal keys (Section 4).
- *Coincidence-of-wants matching*: an incoming order is settled against up to eight resting orders in one PoolManager unlock, swapping the two escrows directly with no protocol fee, with partial fills resting on the book and bounded continuation by anyone (Section 5).
- *Application-specific sequencing*: each pool chooses FIFO or USDC priority bids to order same-price orders, where bids are escrowed at creation, non-refundable, and donated to the liquidity providers active when the order fills (Section 6).
- Two *fallbacks* that keep the book liquid without a market maker: the pool's own liquidity, tried all-or-nothing against the remainder, and permissionless fillers who settle at the maker's live minimum plus a fee (Sections 7 and 8).
- A *machine-checked specification* of the escrow, conservation, backing, price-time-priority and non-negativity properties, maintained against the source on every commit (Section 11).

The design is deployed as a test deployment on Arc and, in its original single-order form, on Unichain. It is not audited and not launched; Section 14 says exactly what is live.

2 Background: Uniswap v4

Uniswap v4 [2] replaces per-pool contracts with a single `PoolManager` that holds all pools' reserves and settles every interaction inside an `unlock` callback. Within the callback a caller accumulates signed currency deltas by swapping, modifying liquidity, donating, or minting and burning ERC-6909 claims; the unlock succeeds only when every delta has been settled to zero by transferring tokens in or taking them out. Claims are the PoolManager's internal ledger: a balance of claim tokens is a right to withdraw that currency, and moving claims between addresses moves value without moving ERC-20 tokens.

A pool may be initialised with a hook contract whose callbacks run before and after swaps, liquidity changes and donations. With the *before-swap-returns-delta* permission, a hook's `beforeSwap` may return a custom delta that the PoolManager applies in place of, or in addition to, the pool's own swap. Returning a delta that consumes the entire exact input leaves nothing for the curve to swap: the pool's state is untouched, the caller owes the input, and the hook decides what the input was for. Async Swap is built on that permission. Pools using the hook are initialised with the dynamic-fee flag, which lets the hook set the LP fee per swap, and must contain the chain's configured USDC on one side.

3 Orders

3.1 Definition

An order is the tuple

$$\mathit{AsyncOrder} = (\mathit{key}, \mathit{owner}, \mathit{zeroForOne}, \mathit{amountIn}, \mathit{amountOutMin}, \mathit{priorityFee}, \mathit{sqrtPrice}, \mathit{salt})$$

where *key* is the Uniswap v4 pool key (the two currencies, the fee flag, the tick spacing and the hook), *owner* is the maker, *zeroForOne* is the direction, *amountIn* is the exact input, *amountOutMin* is the minimum output the maker accepts for that input, *priorityFee* is the USDC bid discussed in Section 6, *sqrtPrice* is carried for the v4 swap call and otherwise unused, and *salt* is a 32-byte value chosen by the maker. The order's identity is

$$\mathit{orderId} = \text{keccak256}\big(\text{abi.encode}(\mathit{chainId}, \mathit{poolId}, \mathit{owner}, \mathit{zeroForOne}, \mathit{salt})\big),$$

so a maker knows its order id before sending the transaction. If the id is already in use the hook walks $\mathit{salt} \leftarrow \text{keccak256}(\mathit{salt})$ to a free slot and reports the final salt in the creation event.

The price of an order is the ratio of its two amounts. Because *amountIn* is exact and *amountOutMin* is a floor, an order is a commitment to trade the whole input at that ratio or better; it is never a market order, and it never trades at a worse price than its owner wrote down.

3.2 Creation and escrow

A maker creates an order through the Router, directly with `swap(order, userData)` or, for ERC-20 input, with `swapWithSig(order, permit, grant)`: an EIP-712 signature over a Permit2 witness that binds the maker, the two tokens, the input, the minimum, the bid and the pool, which a relayer may submit on the maker's behalf. Either way the Router opens an unlock, calls the PoolManager's `swap` with the exact input and the order as hook data, and the hook's `beforeSwap` does three things: it validates the order and the Router's authorization for this maker and pool, it mints the input amount as ERC-6909 claims to itself, and it adds the order to the pool's book. The returned delta consumes the whole input, so the pool's curve is never touched. The Router then settles the maker's input to the PoolManager, from the maker's wallet, from Permit2, or as forwarded ETH, and the unlock closes with every delta at zero. From this point the input is held by the PoolManager and claimed by the hook on the maker's behalf: the *escrow*.

Authorization is a separate, revocable grant. A maker names an *executor* per pool, normally the Router, that may create, fill and cancel on its behalf; the grant can be given directly or by signature, and the first signed order on a pool may carry it. Executor authorization is orthogonal to any one order's lifecycle.

4 The order book

4.1 Price keys

Every resting order is indexed on its pool's book by a 128-bit *price key*, a truncated WAD ratio of its two amounts:

$$k = \begin{cases} \left\lfloor \dfrac{amountIn \cdot 10^{18}}{amountOutMin} \right\rfloor & \text{buy side (the maker escrows the numeraire)} \\ \left\lfloor \dfrac{amountOutMin \cdot 10^{18}}{amountIn} \right\rfloor & \text{sell side} \end{cases} \tag{1}$$

A ratio that rounds to zero maps to one, and a ratio above $2^{128} - 1$ saturates. Keys order the tree; two orders with different exact prices can share a key, so keys alone do not decide whether two orders trade.

4.2 A radix tree of price levels

The book is a Patricia radix tree over price keys, held in the hook's storage, with one level per distinct key and a doubly linked list of orders at each level. The tree answers the only question matching needs, *what is the best opposing price*, in a bounded number of storage reads, and inserting or removing a level does not touch the others. Node identifiers are 32-bit and recycled through a free list when a level empties, so the identifier counter tracks the peak number of simultaneously stored prices rather than the number of prices ever seen.

4.3 Crossing

A buy with key k_b and a sell with key k_s cross when the buyer is willing to pay at least what the seller asks. Since truncation is monotone, $k_b > k_s$ is a cross and $k_b < k_s$ is not. Equal keys are decided at full precision:

$$\text{cross} \iff \frac{buyIn}{buyOut} \geq \frac{sellOut}{sellIn} \iff \left\lfloor \frac{buyIn \cdot sellIn}{sellOut} \right\rfloor \geq buyOut, \tag{2}$$

implemented with a full-width multiply-divide. An exact tie counts as a cross. Minimums are never zero, so the divisor is never zero, and at equal keys the quotient is on the order of *buyOut*, so the division cannot overflow for escrowable amounts. Every genuine cross, including the tie, produces a non-zero match that pays each maker at least its signed minimum for the input consumed; this was checked exhaustively for small amounts and on randomised near-tie pairs.

5 Matching by coincidence of wants

When a new order is created, the Router calls the hook's `matchNewOrder` in the same unlock. The matching library reads the best opposing level from the tree and, while the incoming order crosses it, settles the pair: it computes the amounts each side gives and receives at a price that satisfies both minimums, burns the corresponding escrow claims, and transfers the tokens to the two makers. ERC-20 proceeds are transferred directly; native proceeds are minted as PoolManager claims that the recipient redeems, so a recipient that rejects ETH can never block a match. A match swaps the two escrowed inputs with no protocol fee, so rounding can never move value to or from the protocol.

Matching is bounded to eight pairs per transaction, which keeps the gas of creating an order predictable regardless of how deep the opposing side is. If the incoming order is still open after eight matches, or if an equal-key pair that does not cross stops the walk, the order rests with its remainder and anyone may call `Router.matchBook(key, n)`, $1 \le n \le 8$, to continue. Partial fills stay on the book with their remaining input and remaining minimum; input left over after a maker's minimum has been met is returned to that maker.

Within a level, orders are matched in *sequence*, the ordering defined in Section 6. A reprice, or a partial fill that changes an order's price key, re-enters the order at the back of its new level with a fresh sequence number, so no order can jump a queue by editing itself.

6 Application-specific sequencing

Price-time priority is the convention of every central limit order book: better price first, earlier arrival first at the same price. On a blockchain, "earlier" is decided by whoever builds the block, and the right to be first at a price is sold on a market the application does not control. Async Swap lets each pool decide how same-price orders are sequenced, and gives it a way to sell that right itself.

Every pool has an *ordering mode*. In **FIFO** mode, orders at one price match in the order they were sequenced, and *priorityFee* must be zero. In **priority-bid** mode, every order carries a USDC bid of at least 0.10 USDC (10^5 units of six-decimal USDC); a larger remaining bid matches first at the same price, and equal bids fall back to sequence. Both modes are expressed by one sort key per order,

$$priorityKey = (bid \ll 64) \,|\, (2^{64} - 1 - sequence), \tag{3}$$

so the highest bid and, within it, the lowest sequence number sorts first.

A bid is escrowed as PoolManager claims in the same unlock that creates the order. It is *non-refundable*. As the order's input is consumed by a match, a proportional share

$$settled = \left\lfloor \frac{remainingBid \cdot inputUsed}{remainingInputBeforeFill} \right\rfloor \tag{4}$$

is settled, and when the order terminates, by a full fill, a cancellation or a fill from pool liquidity, the residual bid is settled too. A settled bid is donated through `PoolManager.donate` to the liquidity positions in range at the current tick, in the same unlock as the order operation, and goes to the protocol FeeVault when the pool has no active liquidity. A partial fill can leave a bid below the creation minimum; its remaining bid determines its new rank. Repricing assigns fresh time priority without refunding the bid.

Two things a bid does *not* do deserve emphasis. It does not buy inclusion in a block or a position within one: builders order transactions as they always have, and a bid only orders matching inside

this book once the transaction runs. And it does not guarantee execution: an order with the highest bid at a price still waits for a crossing counterparty. What the bid buys is the first claim on the next coincidence of wants at that price, and the payment goes to the liquidity providers who keep the pool quoting rather than to the block builder.

7 Pool liquidity as fallback

Pools using the hook can hold ordinary Uniswap v4 liquidity positions. After order-to-order matching, the hook tries an incoming order's remainder against that liquidity through an external self-call that performs a pool swap for the remaining exact input with the LP fee temporarily set to 30 basis points. The attempt succeeds only if the pool consumes the whole remaining input and returns at least the order's remaining minimum output; otherwise the self-call reverts and the surrounding transaction catches it, so only the liquidity attempt rolls back and the prior book matches and the open remainder survive. Because the swap originates from the hook, the hook's own callbacks are skipped and no second order is created. Anyone may retry a resting order against pool liquidity with `Router.fillFromPoolLiquidity(order)` after the pool price or liquidity moves; a retry does not require the order to be first in the book. These fills pay the 30 basis point fee to active positions; liquidity providers bear the usual inventory and price risk.

8 Fillers

What the book and the pool cannot match can be settled by anyone. A filler calls the Router's `fillOrder` with the order, a price cap and the recipient, funds the output in the same unlock, and receives the order's full remaining input. The maker's minimum is net: the filler pays gross output

$$gross = amountOutMin + \text{calculatePoolFee}(amountOutMin), \qquad \text{calculatePoolFee}(n) = \left\lfloor \frac{3n}{997} \right\rfloor, \tag{5}$$

which is 0.3% of gross, rounded down; the maker receives exactly its minimum, the FeeVault receives the fee as claims, and the filler receives the escrowed input. The *maxPrice* argument caps what the filler will pay, so a reprice between the filler's quote and its transaction reverts rather than overpays. A fill is all-or-nothing on the order's remainder; partial fills happen only through book matching. A fill requires an authorized executor, normally the Router, and like every other transition it reverts cleanly: a failed fill leaves the order open and the accounting untouched.

A filler is any address with the output token and a reason to want the input: a market maker running inventory, a solver routing the output through another venue, an arbitrage bot, or an AI agent reading the public order feed. The spread between the maker's limit and the market is the filler's reward; the protocol routes nothing on the filler's behalf.

9 Repricing and cancellation

A maker may *reprice* an open order, replacing its minimum output with any nonzero value, directly or by an EIP-712 signature against the hook's domain. The order keeps its escrow and its id, and re-enters the book at the back of its new level. A maker, or its authorized executor, may *cancel* an open order, directly or by signature against the Router's domain: the remaining input is returned in full, the residual bid is settled, and the record is deleted. Cancellation carries no fee.

Fill and cancel delete the order record; their outcomes survive as events. An indexer turns the creation, price-update, match, fill and cancel events into a lifecycle feed, but the contracts neither require nor trust it: the only authoritative view of an order is the hook's storage.

10 Fees

Event	Fee	Paid to
Order-to-order match	none	–
Fill from pool liquidity	30 bps LP fee	active v4 positions
Filler fill	0.3% of gross output	protocol FeeVault
Priority bid (priority pools)	$\geq$ 0.10 USDC, non-refundable	active LPs, else FeeVault
Creation, reprice	none	–
Cancellation	none; input returned	–

11 Properties and verification

The protocol has a Lean 4 [4] and Mathlib specification that models the hook, the Router, the PoolManager's accounting (per-locker deltas that must net to zero, ERC-6909 claims, settle, take, mint, burn, and the before-swap delta), the price tree, the native-ETH claim ledger and the EIP-712 and Permit2 hashing. Every theorem is machine checked with no `sorry` and no additional axioms, and a swarm of agents re-pins the specification to the source and re-proves it on every protocol commit, opening a pull request to the proofs repository only when the build is clean. Properties proved against the current source include:

- **Escrow.** The hook's claim balances equal the sum of open orders' remaining inputs plus escrowed bids, after every entry point.
- **Backing.** The PoolManager's token balance equals claim supply plus pool reserves.
- **Conservation.** Per-currency totals over all ledgers are preserved by every entry point; no match, fill or cancel creates or destroys tokens.
- **Price-time priority.** The level matched is the best price; within a level the order with the higher bid matches first and equal bids match in sequence; a reprice re-enters last.
- **Non-negativity.** Claim balances, and wallet balances outside the PoolManager, never go negative.
- **Bounds.** Stated input bounds under which no intermediate exceeds 2^{256}, and under which 32-bit node identifiers cannot be exhausted without 2^{31} price levels stored at once.
- **Signature binding.** A Permit2 witness commits to exactly the listed fields, so a signed order cannot be replayed with a different direction, size, minimum or pool.

The specification work has found contract bugs as well as proved properties. A rebuild of the protocol from the proofs alone, run against the source, found that a FIFO reprice left a stale `next` pointer in the price-level list, which could make a crossed book unmatchable; node identifiers were never recycled, so a tree would eventually refuse new levels; and the price-key computation could revert for inputs above roughly 10^{58} units. The first two are fixed; the third is stated as an input bound. Alongside the proofs, the source carries a Foundry suite of 144 tests including stateful invariant tests over outstanding escrow, asset conservation and best-price selection, and fork tests against Unichain mainnet state.

12 Security considerations

Orders are public. Every order is visible in contract storage and in events before it is filled. A maker that would be harmed by that visibility should not post; private orders are future work (Section 15).

Bids and block builders. A priority bid orders matching inside the book; it does not interact with transaction ordering. A builder can still sequence two creating transactions as it pleases, and the one that runs first matches first. What the bid removes is the incentive to pay the builder for that position: a maker that wants the next coincidence of wants at a price pays the pool's liquidity providers instead, and the payment is visible and bounded.

Races. Two fillers competing for one order, or a filler and a crossing order, are resolved by whichever transaction runs first; the loser reverts with its funds untouched. The *maxPrice* cap and the all-or-nothing fill protect a filler from paying more than it quoted.

Identity. Order ids bind the chain, pool, owner, direction and salt. Because fill and cancel delete the record, an id can in principle be reused by a later order with the same salt; the maker chooses the salt, and the hook walks to a free one, so reuse requires the maker's own cooperation. Legacy deployments derive ids differently and must not be recomputed with the current formula.

Status. The contracts are not audited. They are deployed for testing and bug discovery, and nothing in this paper is an invitation to trade size on them.

13 Related work

CoW Protocol [3] popularised coincidence-of-wants settlement on-chain, with signed intents collected off-chain and settled in batches by competing solvers. Async Swap keeps the order and the matching on-chain, trades continuously rather than in batches, and needs no solver for a match to happen. UniswapX [5] settles signed intents through a Dutch auction among fillers; its orders are off-chain until filled, and price discovery happens in the auction rather than on a book. Async Swap's fillers play a similar role for the remainder the book cannot match, but at a price the maker fixed rather than a decaying one. On-chain order books on general-purpose chains have historically been considered too expensive to maintain; the radix-tree design, the bounded matching and the reuse of the PoolManager's claim ledger as the escrow are what make this one fit inside a hook.

14 Deployment status

The protocol described here is deployed on Arc (chain id 5042) as a test deployment with book matching, priority bids, partial fills and the pool-liquidity fallback. The original single-order protocol, in which an order rests until a filler settles it in full and there is no book matching, is deployed on Unichain (chain id 130) and remains readable, fillable and cancellable but closed to new orders. Ethereum mainnet has no deployment. Addresses and fees should be read from the contracts and from the application's published machine-readable documents rather than from this paper, as test deployments may be replaced before launch. A mainnet deployment is blocked pending security review.

15 Future work

- **Agent accounts.** Explicit wallet authorization, bounded spend, key isolation, revocation and audit for AI agents acting as makers and fillers; the prerequisite for everything below.
- **Agent-to-agent fills.** Agents filling each other's orders directly, including negotiated fills, through an agent-to-agent endpoint.
- **Private orders and peer-to-peer settlement.** Orders visible only to chosen counterparties, and direct transfers of tokens and orders between parties settled through the protocol.
- **Multi-hop with USDC accounting.** Orders that route through several pools with each hop valued in USDC up to the final token.
- **Any-token pairs.** Lifting the requirement that one side of every pool is USDC.
- **Audit and launch.** Security review of the matching and settlement architecture, then the official deployment.

References

[1] Hayden Adams et al. *Uniswap v3 Core*. Mar. 2021. URL: https://uniswap.org/whitepaper-v3.pdf.

[2] Hayden Adams et al. *Uniswap v4 Core*. Aug. 2024. URL: https://github.com/Uniswap/v4-core/blob/main/docs/whitepaper/whitepaper-v4.pdf.
[3] CoW DAO. *CoW Protocol Documentation*. 2024. URL: https://docs.cow.fi/.
[4] Leonardo de Moura and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language". In: *Automated Deduction – CADE 28*. Springer, 2021.
[5] Uniswap Labs. *UniswapX*. 2023. URL: https://docs.uniswap.org/contracts/uniswapx/overview.